COMSATS University, Islamabad — Pakistan

MediFusion Vision

Implementation Documentation

By

AIZA KHADIM CIIT/FA22-BCS-010/ISB

AREEBA NIAZI CIIT/FA22-BCS-014/ISB

AREEHA NAYAB CIIT/FA22-BCS-015/ISB

Supervisor: Dr. Rasool Bukhsh

Bachelor of Science in Computer Science / Software Engineering (2022–2026)

COMSATS University, Islamabad Pakistan

MediFusion Vision

By

AIZA KHADIM CIIT/FA22-BCS-010/ISB

AREEBA NIAZI CIIT/FA22-BCS-014/ISB

AREEHA NAYAB CIIT/FA22-BCS-015/ISB

Supervisor

Dr. Rasool Bukhsh

Bachelor of Science in Computer Science / Software Engineering (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

Table of Contents

Chapter 3: Design and Architecture

This chapter presents the architectural design of the MediFusion Vision system as implemented. It describes how the AI diagnostic components, backend services, and user-facing web modules are structured and interact. The chapter details the system architecture, the selected architectural pattern, UML-based design models, and the data design used to represent how medical images, reports, user records, and analytical outputs flow through the system.

3.1 System Architecture Overview

The MediFusion Vision system is built on a four-layer architecture: a Frontend Layer (React web application), a Backend/API Layer (Node.js + Express), an AI Engine Layer (Python Flask microservices), and a Database/Storage Layer (PostgreSQL via Sequelize ORM).

The **Frontend Layer** is a React.js single-page application (Vite build system) that provides dedicated dashboards for Patients, Doctors, and Administrators. It communicates with the backend over HTTP REST APIs, supports English/Urdu language switching, and includes web-based accessibility tools (text-to-speech, voice commands, high-contrast mode, large text). Real-time chat during consultations is handled via Socket.IO.

The **Backend/API Layer** is built with Node.js and Express.js, following a Controller–Route–Model structure. It manages authentication (JWT), role-based access control (RBAC), appointment scheduling, scan management, AI orchestration, payment processing via Stripe, in-app notifications, and virtual consultation coordination via Jitsi. It also integrates the Groq AI cloud SDK for consultation transcription and note generation.

The **AI Engine Layer** consists of two independent Python Flask microservices:

- **Brain Model Service** (port 5003): Runs two PyTorch models — ResNet-18 for brain tumor classification and DenseNet-121 for Alzheimer's stage classification. Both produce GradCAM heatmaps with anatomical region analysis.
- **Retinal Model Service** (port 5002): Runs EfficientNetB3 (via TensorFlow/Keras) for retinal disease classification. Produces GradCAM heatmaps with optic zone analysis.

The **Database/Storage Layer** uses PostgreSQL accessed through Sequelize ORM. Medical scan images are stored on the local filesystem via Multer. The database holds all relational data including users, appointments, scans, reports, payments, consultation logs, and notifications.

3.1.1 Purpose

This section provides a conceptual understanding of the MediFusion Vision system architecture as built, showing how the web client, backend API, AI Flask services, database, and third-party integrations (Stripe, Jitsi, Groq) work together to deliver an integrated, AI-powered healthcare platform. It helps stakeholders understand the placement of each module and how data flows across the system — from a patient uploading a scan to receiving a doctor-verified AI diagnostic report.

3.2 Conceptual Architecture Diagram

The system operates across four tiers:

3.2.1 Technologies and Services

Table 3.1: Technologies, Components, and Security Mechanisms

Component	Description	Technology	Security Mechanism
Frontend App	Patient, doctor, admin interfaces for scan upload, reports, appointments, dashboard	React.js (Vite)	Input validation, role-protected routes
Backend API	System logic, AI call orchestration, appointments, notifications, payment routing	Node.js + Express.js	JWT authentication, RBAC, rate limiting
Database Layer	Users, reports, appointments, scans, payments, consultation records	PostgreSQL + Sequelize ORM	Role-based access via ORM queries
Admin Panel	User management, doctor verification, AI stats, finance overview, system settings	React.js (admin routes)	Admin-only middleware `requireRole(['admin'])`
Authentication Service	Verifies identity and roles of patients, doctors, admins using tokens	JWT (jsonwebtoken)	Access token + refresh token, bcrypt hashing
Brain AI Service	Tumor classification (4 classes) and Alzheimer's staging (4 stages) via CNN	PyTorch (ResNet-18, DenseNet-121)	Isolated microservice, no direct DB access
Retinal AI Service	Retinal disease classification (4 classes) via CNN	TensorFlow/Keras (EfficientNetB3)	Isolated microservice, no direct DB access
XAI Engine	GradCAM heatmaps + region-level text explanations for both model services	pytorch-grad-cam, TF GradientTape	Included inside each Flask service
AI Note-Taker	Transcribes doctor-patient consultation audio and generates	Groq SDK (Whisper-large-v3-turbo + LLaMA-3.3-70B)	API key secured in server .env

Component	Description	Technology	Security Mechanism
	structured clinical notes		
Notification Service	In-app notification bell, email alerts for appointments and scan results	Sequelize Notification model + Nodemailer	Role-scoped access to own notifications
Payment Gateway	Handles appointment fee payments, refunds, and transaction history	Stripe (v20.0.0)	PCI-DSS via Stripe, webhook signature verification
Real-Time Chat	Live messaging during virtual consultations	Socket.IO (v4.8.1)	Room-scoped socket connections
Virtual Consultation	Secure video consultations between patient and doctor	Jitsi Meet (embedded iframe)	Appointment-gated meeting links
File Storage	Medical scan images stored on server filesystem	Multer (v2.0.2) + diskStorage	File type/size validation before storage

3.3 Architecture Style / Pattern

MediFusion Vision uses a **Layered Architecture** combined with an **MVC-aligned modular structure**. This ensures clean separation of concerns between the user interface, application services, AI diagnostic engine, and data storage layers.

1. Presentation Layer (Frontend — React.js Web)

Provides role-based interfaces for Patients, Doctors, and Administrators:

- Patient: dashboard, appointment booking, scan upload, scan results with AI heatmaps, medical records, payment history, virtual consultation room
- Doctor: schedule management, patient list, diagnostics interface (AI scan analysis + GradCAM viewer), physical and virtual consultation rooms, profile completion
- Admin: dashboard with live analytics charts, user management (block/unblock/delete), doctor verification queue, AI model stats, finance and payment history, system configuration

The frontend communicates with the backend via REST APIs over HTTP. Multi-language support (English/Urdu) is provided through `react-i18next`. Accessibility tools (TTS, voice commands, high-contrast, large text) are toggled via `AccessibilityTools.jsx` and persisted via the backend patient settings API.

2. Application Layer (Backend + AI Microservices)

The backend follows MVC:

Models — Sequelize ORM models defining the database schema for all 15 entities (User, Patient, DoctorProfile, Appointment, Scan, Report, Payment, PaymentTransaction, ConsultationNote, ChatLog, MedicalHistory, Prescription, AIFeedback, Notification, PatientProfile).

Controllers — Implement all business logic:

- `authController`: registration, login, token refresh, email verification, password reset
- `adminController`: stats, analytics, doctor verification, user block/delete
- `appointmentController`: booking, rescheduling, cancellation, slot availability, no-show marking
- `scanController`: file upload, AI analysis orchestration (routes to brain/retinal Flask services), result storage
- `consultationController`: chat history, message sending, audio transcription, AI note generation, note finalization
- `doctorController`: doctor search, profile management, availability
- `paymentController`: Stripe intent creation, confirmation, refunds, receipts, history
- `notificationController`: get, mark-read, mark-all-read
- `patientSettingsController`: accessibility and language preference persistence
- `reportController`: report creation and retrieval
- `pdfController`: PDF report generation

Routes — Map API endpoints to controller functions with JWT auth and RBAC middleware applied per route.

Services / Utilities:

- `aiNoteService.js`: Groq Whisper transcription + LLaMA-3.3-70B structured note generation
- `emailService.js`: Nodemailer-based email notifications
- `notificationHelper.js`: creates in-app Notification records across controllers

3. Data Layer (PostgreSQL + Local File Storage)

All relational data stored in PostgreSQL via Sequelize ORM. Scan images stored under `server/uploads/` using Multer disk storage. File type (JPEG/PNG only) and size (50 MB max) are validated before storage.

3.4 Design Models

Activity Diagrams

Activity diagrams were created for the core workflows:

1. AI Diagnostic Process

Patient uploads a scan through the Diagnostics page. The backend validates the file (type, size), stores it via Multer, and forwards the image to the appropriate Flask service — Brain Model (port 5003) for MRI or Retinal Model (port 5002) for retinal scans. The Flask service preprocesses the image (resize to 224×224, normalize), runs inference through the CNN model, generates a GradCAM heatmap, performs region analysis, and returns prediction class, confidence score, all class probabilities, base64 heatmap, and textual XAI explanation. The backend stores the result in the Scan record. The doctor views results, verifies or flags the AI diagnosis, and the result is linked to the patient's scan history.

2. Appointment Booking Process

Patient searches doctors by specialization or name. Available time slots are fetched from the doctor's schedule. Patient selects slot, provides reason and appointment type (physical/virtual). System checks for conflicts, creates the appointment in `pending_payment` state, initiates a Stripe PaymentIntent, and on successful payment confirmation, moves the appointment to `booked` state and dispatches in-app + email notifications to both patient and doctor.

3. Virtual Consultation Process

At the scheduled appointment time, patient and doctor navigate to the Consultation Room page. The system generates a Jitsi meeting link scoped to the appointment ID. Both parties join the video call. The doctor can start the AI Note-Taker, which records audio, sends it to `consultationController.uploadAudioAndGenerateNotes`, which calls Groq Whisper for transcription (handling mixed Urdu/English) and then LLaMA-3.3-70B to produce structured clinical notes (chief complaint, HPI, assessment, plan, patient summary). The doctor reviews and edits the generated notes before finalizing.

4. Doctor Registration Process

Doctor registers with role `doctor`, submits qualification details and PMDC license number. Account is created in `pending_verification` status. Admin receives a notification and reviews the pending application in the Doctor Verification Queue. Admin approves or rejects — approved doctors gain full access to the doctor dashboard; rejected doctors are notified with the reason.

5. Scan Management Process

Patient or admin navigates to the upload page, selects scan type (MRI or retinal), and uploads a JPEG/PNG file. Multer middleware validates file format and enforces the 50 MB size limit. On success, the scan record is created in the database linked to the patient, and the file is stored in `server/uploads/`. The scan appears in the patient's My Scans page and in the doctor's Diagnostics interface for analysis.

6. Payment Processing Process

At appointment confirmation, the frontend calls `POST /api/payments/create-intent` to generate a Stripe PaymentIntent. The Stripe.js `CardElement` on the frontend collects card details without them ever touching the server. On submission, `confirmPayment` is called, Stripe processes the transaction, and a webhook (`POST /api/payments/webhook`) confirms the final status. On success, a Payment and PaymentTransaction record are created, the appointment moves to `booked`, and a receipt is available via `GET /api/payments/:id/receipt`.

7. Admin System Monitoring Process

Admin logs in and lands on the Admin Dashboard which fetches real-time stats (`/api/admin/stats`) and time-series analytics (`/api/admin/analytics`) including monthly revenue, patient registrations, and doctor registrations rendered as Chart.js line charts. Admin can navigate to User Management (view, block, delete), Doctor Verification (approve/reject pending doctors), Medical AI Management (brain and retinal scan counts, flagged scans, model accuracy display), Finance (full payment transaction history), and System Settings (platform configuration).

Class Diagram

The system is built around a User base entity with Patient and Doctor specializations. Core relationships:

- **User** (base): userId, email, passwordHash, name, phone, role, is_active, createdAt
- **Patient** → extends User: date_of_birth, gender, blood_group, emergency_contact (via PatientProfile)
- **DoctorProfile** → extends User: specialization, pmdc_number, years_experience, consultation_fee, availability (JSON), status (pending/approved/rejected)
- **Appointment**: appointmentId, patient_id (FK User), doctor_id (FK DoctorProfile), dateTime, type (physical/virtual), status, reason, meeting_link, fee
- **Scan**: scanId, patient_id (FK), doctor_id (FK), scan_type (brain/retinal), file_path, upload_source, ai_result (JSON), ai_confidence, status, gradcam_heatmap (base64), ai_explanation (JSON)
- **Report**: reportId, scan_id (FK), doctor_id (FK), findings, final_diagnosis, doctor_notes, status (draft/finalized)

- **ConsultationNote**: noteId, appointment_id (FK), doctor_id (FK), transcript, structured_notes (JSON), finalized
- **ChatLog**: chatId, appointment_id (FK), sender_id (FK), message, timestamp
- **Payment**: paymentId, appointment_id (FK), patient_id (FK), amount, stripe_payment_intent_id, status
- **PaymentTransaction**: transactionId, payment_id (FK), stripe_charge_id, payment_method, status, amount
- **Notification**: notificationId, user_id (FK), title, message, type, is_read, createdAt
- **MedicalHistory**: historyId, patient_id (FK), condition, medication, allergy, notes
- **AIFeedback**: feedbackId, scan_id (FK), doctor_id (FK), ai_prediction, corrected_diagnosis, comments

---

Sequence Diagrams

1. AI Diagnostic Execution

```
Doctor → Diagnostics Page → POST /api/scans/upload (multipart)
  Backend → Multer → validates file → saves to disk → creates Scan record
  Backend → POST http://localhost:5003/api/analyze (Brain) or POST
http://localhost:5002/api/analyze (Retinal)
    Flask Service → preprocess image → CNN inference → GradCAM → region analysis → JSON
response
  Backend → updates Scan.ai_result, Scan.gradcam_heatmap, Scan.ai_explanation
  Backend → 200 OK with full AI result → Doctor sees prediction + heatmap + XAI text
  Doctor → POST /api/scans/:id/feedback (verify/flag/correct)
```

2. Appointment Booking with Payment

```
Patient → BookAppointment Page → GET /api/doctors (search)
Patient → GET /api/appointments/slots?doctorId=X&date=Y
Patient → POST /api/appointments/book { doctorId, slotTime, type, reason }
  Backend → checks slot availability → creates Appointment (status: pending_payment)
  Backend → POST /api/payments/create-intent { appointmentId }
    Backend → Stripe API → creates PaymentIntent → returns client_secret
  Patient → Stripe.js confirmCardPayment(client_secret) → Stripe processes card
  Stripe → POST /api/payments/webhook (charge.succeeded)
    Backend → updates Payment to succeeded → updates Appointment to booked
    Backend → creates Notification for patient + doctor
    Backend → sends email via emailService
```

3. Virtual Consultation with AI Note-Taker

```
Patient + Doctor → ConsultationRoom page (appointment must be booked)
  Page generates Jitsi room URL scoped to appointmentId
  Both parties join Jitsi iframe video call
Doctor → clicks "Start Recording" → browser MediaRecorder captures audio
Doctor → clicks "Stop & Generate Notes" → POST /api/consultation/:id/notes/generate (audio file)
  Backend → consultationController.uploadAudioAndGenerateNotes
  Backend → aiNoteService.transcribeAudio(audioFile) → Groq Whisper-large-v3-turbo → transcript
  Backend → aiNoteService.generateConsultationNotes(transcript, patientContext)
    → Groq LLaMA-3.3-70B → structured JSON { chiefComplaint, HPI, symptoms, assessment, plan, patientSummary }
  Backend → saves ConsultationNote (draft)
Doctor → edits fields in note modal → POST /api/consultation/:id/notes/finalize
  Backend → marks ConsultationNote.finalized = true → patient can now view patientSummary
```

4. Patient Scan Upload

```
Patient → UploadScan page → selects file + scan type
  POST /api/scans/upload/external (multipart/form-data)
  Backend → Multer.checkFileType validates JPEG/PNG/JPG only
  Backend → scanValidation.js enforces 50 MB limit
  Backend → diskStorage saves to server/uploads/
  Backend → creates Scan record { patient_id, scan_type, file_path, upload_source: 'patient' }
  Backend → 201 Created → patient redirected to My Scans
  Doctor → sees scan in Diagnostics → can run AI analysis
```

5. Doctor Registration Approval

```
Doctor → Register page → POST /api/auth/register { role: 'doctor', pmdc_number, specialization, ... }
  Backend → creates User + DoctorProfile { status: 'pending_verification' }
  Backend → sends welcome email to doctor
Admin → VerificationQueue page → GET /api/admin/pending-doctors
Admin → reviews credentials → POST /api/admin/verify-doctor/:id { action: 'approve' }
  Backend → updates DoctorProfile.status = 'approved', User.is_active = true
  Backend → sends approval notification + email to doctor
Doctor → can now log in and access full doctor dashboard
```

State Transition Diagrams

1. Appointment Object Lifecycle

```
[Slot Available] → book() → [Pending Payment]
[Pending Payment] → payment_success() → [Booked]
[Pending Payment] → payment_failed() / cancel() → [Cancelled]
[Booked] → start_time_reached() → [In Progress]
[Booked] → cancel() → [Cancelled]
[In Progress] → complete() → [Completed]
[In Progress] → no_show() → [No Show]
[Booked] → reschedule() → [Pending Payment] (new slot)
```

2. Doctor Registration Lifecycle

```
[Not Registered] → submit_registration() → [Pending Verification]
[Pending Verification] → admin_approve() → [Approved / Active]
[Pending Verification] → admin_reject() → [Rejected]
[Approved] → admin_block() → [Blocked]
[Blocked] → admin_unblock() → [Approved / Active]
```

3. Diagnosis Report Lifecycle

```
[Scan Uploaded] → run_ai_analysis() → [AI Processing]
[AI Processing] → analysis_complete() → [AI Result Ready]
[AI Result Ready] → doctor_verify() → [Doctor Verified]
[AI Result Ready] → doctor_flag() → [Flagged / Incorrect]
[Doctor Verified] → generate_report() → [Report Finalized]
[Flagged / Incorrect] → doctor_correct() → [Doctor Verified]
```

4. Payment Transaction Lifecycle

```
[Not Initiated] → create_intent() → [Intent Created]
[Intent Created] → card_submitted() → [Processing]
[Processing] → stripe_success() → [Succeeded]
[Processing] → stripe_failure() → [Failed]
[Succeeded] → admin_refund() → [Refunded]
```

3.5 Data Design

The data design of MediFusion Vision uses PostgreSQL with Sequelize ORM for all structured data and the local filesystem for medical image files.

Major Data Entities and Relationships

Entity	Description	Key Fields	Relationships
User	Base entity for all system users	userId, email, passwordHash, name, phone, role, is_active, createdAt	Parent of Patient, Doctor; has Notifications
PatientProfile	Patient-specific demographics	patientId, user_id (FK), date_of_birth, gender, blood_group, emergency_contact	Belongs to User; has Appointments, Scans
DoctorProfile	Doctor credentials and availability	doctorId, user_id (FK), specialization, pmdc_number, consultation_fee, availability (JSON), status	Belongs to User; has Appointments, Scans
Appointment	Scheduled consultation	appointmentId, patient_id, doctor_id, dateTime, type, status, reason, meeting_link, fee	Links Patient and Doctor; has Payment, ConsultationNote
Scan	Medical imaging record	scanId, patient_id, doctor_id, scan_type, file_path, ai_result (JSON), ai_confidence, gradcam_heatmap, status	Links to Patient, Doctor, Report
Report	AI + doctor final diagnosis	reportId, scan_id, doctor_id, findings, final_diagnosis, doctor_notes, status	Belongs to Scan and Doctor
ConsultationNote	AI-scribed clinical notes	noteId, appointment_id, doctor_id, transcript, structured_notes (JSON), finalized	Belongs to Appointment
ChatLog	Real-time consultation messages	chatId, appointment_id, sender_id, message, timestamp	Belongs to Appointment
Payment	Stripe payment record	paymentId, appointment_id, patient_id, amount, stripe_payment_intent_id, status	Belongs to Appointment
PaymentTransaction	Individual transaction detail	transactionId, payment_id, stripe_charge_id, payment_method, status	Belongs to Payment
Notification	In-app notification	notificationId, user_id, title, message, type, is_read, createdAt	Belongs to User

Entity	Description	Key Fields	Relationships
MedicalHistory	Patient medical background	historyId, patient_id, condition, medication, allergy, notes	Belongs to Patient
AIFeedback	Doctor correction of AI result	feedbackId, scan_id, doctor_id, ai_prediction, corrected_diagnosis, comments	Belongs to Scan and Doctor
Prescription	Consultation prescriptions	prescriptionId, appointment_id, doctor_id, medicines, notes	Belongs to Appointment

Data Storage and Processing

Storage:

- Structured data (users, appointments, medical records, payments, notifications) stored in PostgreSQL with ACID compliance via Sequelize
- Medical image files (MRI, retinal scans) stored as JPEG/PNG in `server/uploads/` on the local filesystem with file path reference in the Scan table
- AI model weights stored as `.pth` (PyTorch) and `.h5` (Keras) binary files, loaded once at Flask service startup

Processing:

- All database interactions go through Sequelize model methods and associations
- AI analysis pipelines handle image preprocessing, CNN inference, and GradCAM generation within the Flask services
- Real-time consultation chat is processed through Socket.IO event handlers in `server/index.js`
- Audio-to-notes pipeline runs asynchronously: Multer saves the audio file, `aiNoteService.js` calls Groq API, result is returned in the HTTP response

Security and Privacy:

- All passwords hashed with bcrypt (salt rounds 10) before database storage
- JWT access tokens (short-lived) and refresh tokens used for stateless authentication
- RBAC middleware (`requireRole`) applied at the route level — all admin endpoints require `role === 'admin'`, doctor-only routes require `role === 'doctor'`
- Stripe never sends card data through the Node.js server — Stripe.js handles card collection directly and passes only a payment intent token
- Environment variables (JWT secret, Stripe keys, Groq API key, DB credentials) are stored in `.env` and excluded from version control via `.gitignore`

Chapter 4: Implementation

This chapter presents the implementation details of MediFusion Vision — specifically the components developed as part of this implementation: the React.js web application, the Node.js/Express.js backend, and the two Python Flask AI microservices. Each component was developed using technologies chosen for their suitability to medical AI workflows, with a focus on diagnostic accuracy, secure data handling, and usability.

4.1 Project Methodology & Algorithms

4.1.1 Step-by-Step Approach

Data Collection

What: Three separate medical imaging datasets were used:

1. Brain Tumor MRI Dataset — classes: Glioma, Meningioma, Pituitary, No Tumor
2. Alzheimer's Disease MRI Dataset — classes: MildDemented, ModerateDemented, NonDemented, VeryMildDemented
3. Retinal Disease Image Dataset — classes: Cataract, Diabetic Retinopathy, Glaucoma, Normal

How: Datasets sourced from Kaggle and downloaded in JPEG/PNG format. Class labels were organized into structured directory trees (`train/` , `val/` , `test/` splits). Retinal dataset was pre-labeled with disease category per image.

Why: Labeled imaging datasets are required to train supervised CNN classifiers. MRI datasets capture tissue abnormalities at the structural level; retinal photographs capture vascular and nerve-head pathology, together covering two major diagnostic specialties (neurology and ophthalmology).

Data Preprocessing

Brain Models (PyTorch transforms):

- Resize to 224×224 pixels
- Random horizontal flip + 10° rotation for augmentation (training only)
- Normalize using ImageNet mean `[0.485, 0.456, 0.406]` and std `[0.229, 0.224, 0.225]`
- Tumor model: standard random crop and flip
- Alzheimer model: RandomResizedCrop (scale 0.8–1.0), brightness/contrast jitter, WeightedRandomSampler for class imbalance

Retinal Model (TF/Keras preprocessing):

- Resize to 224×224 pixels
- `preprocess_input` from EfficientNet application (scales to -1 to 1 range)
- Horizontal flip augmentation during training

Why: Medical images vary in brightness, orientation, and scale. Preprocessing enforces uniformity so models learn clinically relevant features rather than image acquisition artefacts.

Feature Extraction

No manual feature engineering was applied. Deep CNN backbones extract hierarchical features automatically:

- ResNet-18 residual skip connections preserve edge and texture features for tumor boundary detection
- DenseNet-121 dense feature reuse captures subtle patterns (cortical thinning, hippocampal changes) relevant to Alzheimer staging
- EfficientNetB3 compound scaling balances depth, width, and resolution for efficient retinal pathology detection

---

AI Model Training

Brain Tumor Model — ResNet-18 (PyTorch):

- Pre-trained on ImageNet; final FC layer replaced with 4-class output
- Optimizer: Adam, LR = 0.0001
- Loss: CrossEntropyLoss
- Epochs: 20 with early stopping (patience = 3 on validation accuracy)
- Model saved on best validation accuracy

Alzheimer Model — DenseNet-121 (PyTorch):

- Trained from scratch with Dropout(0.5) before classifier
- Optimizer: Adam, LR = 1e-4, ReduceLROnPlateau scheduler
- Loss: CrossEntropyLoss with WeightedRandomSampler for class balance
- Epochs: 20 with early stopping (patience = 5)

Retinal Disease Model — EfficientNetB3 (TensorFlow/Keras):

- Pre-trained EfficientNetB3 backbone; top layers replaced with GlobalAveragePooling2D → Dropout(0.3) → Dense(4, softmax)
- Optimizer: Adam
- Loss: Categorical CrossEntropy

- Fine-tuned on retinal disease dataset
- Saved as `.h5` weights file (`efficientnetb3-Eye Disease-91.47.h5`)

GradCAM Explainability Implementation

GradCAM was implemented independently in each Flask service:

Brain Model (pytorch-grad-cam library):

- Tumor target layer: `resnet18.layer4[-1].conv2`
- Alzheimer target layer: `densenet121.features.norm5`
- After inference, GradCAM produces a grayscale activation map (224×224)
- `analyze_gradcam_regions()` divides the map into 5 anatomical zones: frontal lobe (rows 0–75), left temporal (rows 50–174, cols 0–112), right temporal (rows 50–174, cols 112–224), central (rows 75–150), posterior/parietal (rows 150–224)
- Mean activation per zone is computed; top two zones are extracted
- `generate_explanation()` maps zone names to clinical significance per disease (e.g., frontal lobe + pituitary tumor → sella turcica region, right temporal + glioma → right temporal lobe infiltration)
- GradCAM overlay (heatmap superimposed on original image) is returned as base64 PNG

Retinal Model (TensorFlow GradientTape):

- `make_gradcam_heatmap()` : uses `tf.GradientTape` to compute gradient of top predicted class score with respect to the last convolutional layer output
- Heatmap is upsampled to image size via bilinear interpolation
- Five anatomical zones used: optic_disc, macula, superior, inferior, periphery
- `DISEASE_KNOWLEDGE` dictionary maps each class to its expected zones and clinical interpretation (e.g., glaucoma → optic disc cupping; diabetic retinopathy → microaneurysms in macula + periphery; cataract → central lens opacity)
- Zone activation scores computed; result includes whether model focused on medically expected regions

---

AI Note-Taker Pipeline

A two-step pipeline using Groq cloud AI was implemented in `server/utils/aiNoteService.js` :

Step 1 — Transcription: `transcribeAudio(audioFilePath)` calls `groq.audio.transcriptions.create({ model: 'whisper-large-v3-turbo', file, language: 'en' })`. Handles mixed Urdu/English input — the system prompt explicitly instructs the model to

translate all Urdu content to professional medical English before structuring notes.

Step 2 — Structured Note Generation: `generateConsultationNotes(transcript, patientContext)` calls `groq.chat.completions.create({ model: 'llama-3.3-70b-versatile', temperature: 0.2 })` with a medical scribe system prompt that enforces JSON output containing: `chiefComplaint`, `HPI` (history of present illness), `symptoms[]`, `assessment`, `plan`, `followUp`, `patientSummary` (layman-friendly). Patient context (name, age, gender, medical history) is injected into the prompt for personalized notes.

4.1.2 Algorithms

Table 4.1: Core Algorithms in MediFusion Vision

Algorithm	Input	Output	Description
CNN Inference (Brain)	224×224 RGB image	class label, confidence, all-class probabilities	Image passed through ResNet-18 (tumor) AND DenseNet-121 (Alzheimer) in single request; both results returned together
CNN Inference (Retinal)	224×224 RGB image	class label (cataract/DR/glaucoma/normal), confidence, all-class probs	Image passed through EfficientNetB3; softmax probabilities returned
GradCAM — Brain	Image tensor, trained model, target layer, predicted class index	Grayscale activation map (224×224), base64 overlay PNG	Gradient of class score w.r.t. target conv layer activations, pooled spatially, ReLU applied, upsampled
GradCAM — Retinal	Image tensor, last conv layer, predicted class index	Heatmap array, base64 overlay PNG	TF GradientTape records gradients, mean-pooled over spatial dimensions, bilinear upsampled
GradCAM Region Analysis	Grayscale activation map	Top region name, second region name, region scores dict	Anatomical grid overlaid on map; mean activation computed per zone; top two zones extracted
AI Scribe	Audio file path, patient context dict	Structured JSON clinical note	Whisper transcription → LLaMA note generation; bilingual (EN/UR) input supported
Slot Availability Check	doctorId, date	Array of available time slots	Fetches doctor's availability schedule, subtracts already-booked slots for that day
JWT Auth	email, password	Access token, refresh token	bcrypt.compare() verifies password; jwt.sign() creates tokens with role and userId payload

Algorithm Pseudocode — Dual Brain Model Analysis:

```
procedure ANALYZE_BRAIN_SCAN(image_file):
    pil_image ← load_and_convert_RGB(image_file)
    img_tensor ← preprocess(pil_image)  // resize, normalize

    // Tumor Model
    with torch.no_grad():
        tumor_logits ← tumor_model(img_tensor)
        tumor_probs ← softmax(tumor_logits)
        tumor_idx ← argmax(tumor_probs)
        tumor_class ← tumor_classes[tumor_idx]
        tumor_conf ← tumor_probs[tumor_idx] * 100

    // Alzheimer Model
    with torch.no_grad():
        alz_logits ← alz_model(img_tensor)
        alz_probs ← softmax(alz_logits)
        alz_idx ← argmax(alz_probs)
        alz_class ← alz_classes[alz_idx]
        alz_conf ← alz_probs[alz_idx] * 100

    // GradCAM for both models
    tumor_heatmap, tumor_overlay ← generate_gradcam(tumor_model, tumor_target_layer,
img_tensor, tumor_idx, pil_image)
    alz_heatmap, alz_overlay ← generate_gradcam(alz_model, alz_target_layer, img_tensor,
alz_idx, pil_image)

    // Region Analysis
    tumor_xai ← generate_explanation("tumor", tumor_class, tumor_conf, tumor_heatmap)
    alz_xai ← generate_explanation("alzheimer", alz_class, alz_conf, alz_heatmap)

    return {
        tumor: { class: tumor_class, confidence: tumor_conf, all_probs: tumor_probs,
                 heatmap_b64: tumor_overlay, explanation: tumor_xai },
        alzheimer: { class: alz_class, confidence: alz_conf, all_probs: alz_probs,
                     heatmap_b64: alz_overlay, explanation: alz_xai }
    }
end procedure
```

4.2 Training Results & Model Evaluation

Brain Tumor Classification — ResNet-18

Dataset: Brain Tumor MRI Dataset (Kaggle) **Classes:** glioma, meningioma, no_tumor, pituitary **Split:** Pre-split Train / Validation / Test folders **Preprocessing:** Resize 224×224, random horizontal flip, 10° rotation, ImageNet normalization **Training:** Kaggle GPU (NVIDIA Tesla T4), PyTorch, batch size 32, Adam LR 0.0001, 20 epochs (early stopping patience 3), CrossEntropyLoss **Model saved:** Best validation accuracy checkpoint

Test Performance:

Class	Precision	Recall	F1-score	Support
glioma	1.00	0.93	0.96	28
meningioma	0.93	1.00	0.97	28
no_tumor	1.00	1.00	1.00	20
pituitary	1.00	1.00	1.00	28

- **Overall Accuracy: 98%**
- Macro Avg: 0.98 precision, 0.98 recall, 0.98 F1
- Observations: Very high performance with clean class separation; minimal meningioma/glioma confusion expected given visual similarity.

Alzheimer Stage Classification — DenseNet-121

Dataset: Alzheimer's Disease MRI Dataset (Kaggle) **Classes:** MildDemented, ModerateDemented, NonDemented, VeryMildDemented **Split:** Train / Validation / Test **Preprocessing:** RandomResizedCrop (scale 0.8–1.0), horizontal flip, 10° rotation, brightness/contrast jitter, ImageNet normalization, WeightedRandomSampler **Training:** Kaggle GPU (NVIDIA Tesla T4), PyTorch, batch size 32, Adam LR 1e-4, ReduceLROnPlateau scheduler, Dropout 0.5, 20 epochs (early stopping patience 5), CrossEntropyLoss **Model saved:** Best validation accuracy checkpoint

Test Performance:

Class	Precision	Recall	F1-score	Support
MildDemented	0.99	0.99	0.99	896
ModerateDemented	1.00	1.00	1.00	647
NonDemented	0.98	0.97	0.98	960
VeryMildDemented	0.98	0.98	0.98	896

- **Overall Accuracy: 99%**
- Macro Avg: 0.99 precision, 0.99 recall, 0.99 F1
- Observations: Highly stable due to WeightedRandomSampler and augmentation. Model handles the class imbalance well.

Retinal Disease Classification — EfficientNetB3

Dataset: Retinal Eye Disease Dataset (Kaggle) **Classes:** cataract, diabetic_retinopathy, glaucoma, normal **Preprocessing:** Resize 224×224, EfficientNet preprocess_input scaling, horizontal flip augmentation **Training:** TensorFlow/Keras, EfficientNetB3 backbone (ImageNet pretrained), fine-tuned top layers, Adam optimizer, categorical crossentropy loss **Model saved as:** `efficientnetb3-Eye Disease-91.47.h5`

Test Performance:

- **Overall Accuracy: 91.47%**
- The model achieves strong discrimination between retinal conditions, particularly glaucoma (optic disc cupping) and diabetic retinopathy (vascular changes). Some overlap occurs between early cataract and normal cases due to subtle image differences.

4.3 Security Techniques

Authentication:

1. JWT-based stateless authentication — tokens include `userId`, `role`, `iat`, `exp`; short-lived access tokens paired with refresh tokens stored securely
2. Role-Based Access Control (RBAC) via `requireRole` middleware applied at route level:
    - Admin: full system control (`/api/admin/*`)
    - Doctor: own schedule, patient scans, consultations, diagnostics
    - Patient: own appointments, scans, results, payments

Encryption:

1. Passwords hashed with bcrypt (salt rounds = 10) before storing; plaintext passwords never persisted
2. Stripe.js handles card data directly — card numbers never pass through the Node.js server
3. Environment secrets (JWT secret, Stripe keys, Groq API key, DB password) stored in `.env`, excluded from version control

Attack Prevention:

Table 4.2: Attack Prevention Techniques

Threat	Mitigation
XSS (Cross-Site Scripting)	React's JSX escapes output by default; no `dangerouslySetInnerHTML` used
SQL Injection	All database queries use Sequelize ORM parameterized queries — no raw SQL with user input
Brute Force	Route-level rate limiting on auth endpoints; bcrypt cost ensures slow hash comparison
Unauthorized Access	JWT token verification middleware on all protected routes; role checked per endpoint
File Upload Abuse	Multer `checkFileType` restricts to JPEG/PNG/JPG; 50 MB size limit enforced before storage
Session Hijacking	Short-lived JWT tokens; refresh token rotation; HTTPS required in production
Payment Tampering	Payment amount computed server-side from appointment record; client cannot set amount

4.4 External APIs / SDKs

Table 4.3: APIs and SDKs Used

Name & Version	Description	Purpose	Key Functions Used
Stripe v20.0.0	Payment processing platform	Secure appointment fee collection and refunds	`stripe.paymentIntents.create()`, `stripe.refunds.create()`, `stripe.webhooks.constructEvent()`
Socket.IO v4.8.1	Real-time bidirectional communication	Virtual consultation live chat	`io.on('connection')`, `socket.join(room)`, `socket.emit('receive_message')`
Multer v2.0.2	File upload middleware	Medical scan file uploads with validation	`multer.diskStorage()`, `upload.single('scan')`, `checkFileType` filter
bcryptjs v3.0.3	Password hashing library	Secure password storage and verification	`bcrypt.hash()`, `bcrypt.compare()`
jsonwebtoken v9.0.2	JWT token creation and verification	Stateless authentication across all API calls	`jwt.sign()`, `jwt.verify()`
Sequelize v6.37.7	PostgreSQL ORM	Database model definitions, associations, CRUD, migrations	Model definitions, `findAll`, `findOne`, `create`, `update`, associations
Groq SDK (latest)	Groq cloud AI platform	Audio transcription (Whisper) and note generation (LLaMA)	`groq.audio.transcriptions.create()`, `groq.chat.completions.create()`
Nodemailer v6+	Email sending library	Registration confirmation, appointment and scan notifications	`transporter.sendMail()`

Name & Version	Description	Purpose	Key Functions Used
pytorch-grad-cam	GradCAM explainability for PyTorch	Brain model heatmap generation	`GradCAM(model, target_layers)`, `ClassifierOutputTarget`
React Router v7.9.6	Client-side routing	SPA navigation and role-based route protection	`<BrowserRouter>`, `<Routes>`, `useNavigate()`, `<Navigate>`
react-i18next	Internationalization framework	English/Urdu language switching across the web app	`useTranslation()`, `i18n.changeLanguage()`
Framer Motion v12+	Animation library	Smooth page transitions and dashboard animations	`motion.div`, `animate`, `whileHover`, `variants`
Chart.js v4.5.1	Data visualization	Admin dashboard revenue and user growth charts	Line charts, bar charts
Jitsi Meet (embed)	Video conferencing	Virtual consultation video calls	Jitsi IFrame API embedded in ConsultationRoom.jsx
react-hot-toast	Notification toasts	Success/error feedback across all user actions	`toast.success()`, `toast.error()`, `toast.info()`

4.5 User Interface

The MediFusion Vision web UI is developed for three main user groups — Patients, Doctors, and Administrators — using React.js with Tailwind CSS. The design is clean, medically appropriate, and accessibility-focused.

Patient Interface (10 pages):

- *Dashboard* — upcoming appointments, recent scans, quick actions

- *Doctor Search* — search by name or specialization, view profiles and fees
- *Book Appointment* — select doctor → pick available time slot → enter reason → Stripe payment
- *My Appointments* — list of all appointments with status; cancel option for pending
- *Upload Scan* — drag-and-drop file upload with type and format validation
- *My Scans* — paginated list of uploaded scans with AI analysis status
- *Scan Results* — GradCAM heatmap viewer, disease prediction, confidence bars, region-level XAI text, doctor notes
- *Medical Records* — consultation notes (patient summary), medical history
- *Consultation Room* — Jitsi video embed, live chat panel, AI note display
- *Billing / Payment History* — transaction history and receipts

Doctor Interface (8 pages):

- *Dashboard* — today's appointments, patient stats, quick navigation
- *Schedule Management* — weekly availability configuration, view booked slots
- *Patient Management* — full patient list with search
- *Patient Details* — individual patient history, scans, and reports
- *Diagnostics* — scan selector, "Run AI Analysis" button, GradCAM heatmap viewer, confidence breakdown, XAI text, feedback/flag controls
- *Physical Consultation Room* — appointment notes and prescription entry for in-person visits
- *Virtual Consultation Room* — Jitsi video + real-time chat + AI Note-Taker controls
- *Profile Completion* — qualifications, PMDC number, specialization, consultation fee, availability grid

Admin Interface (7 pages):

- *Dashboard* — total users, pending doctors, appointments, revenue; monthly line charts (revenue, patient growth, doctor growth) using live API data
- *User Management* — full user list with search, role filter, block/unblock/delete
- *Doctor Verification* — pending doctor applications with approve/reject
- *Medical & AI Management* — brain and retinal scan counts, flagged scan count, model accuracy display
- *Finance & Support* — payment transaction table with patient, doctor, amount, method, status
- *System & Content* — platform name, support email, maintenance mode toggle (saved to localStorage)
- *Notification Bell* — shared across all layouts; shows unread count badge, marks as read, marks all read

Accessibility Tools (web-wide, patient-facing):

- Text-to-speech toggle (Web Speech API)
- Voice command activation (Web Speech Recognition API)
- High-contrast mode (CSS class toggle on `<body>`)
- Large text mode (CSS class toggle on `<body>`)
- All preferences persisted via `/api/patient/settings`

Multi-Language (web-wide):

- English / Urdu toggle via `LanguageSwitcher.jsx` using `react-i18next`
- UI strings in both languages via translation JSON files
- AI consultation transcription supports mixed Urdu/English input

4.6 Deployment

The system runs as a local development stack with the following configuration:

Component	Technology	Port / Location
Backend Server	Node.js v22 + Express v5.1.0	Port 5000
Web Frontend	React v19 + Vite v7	Port 5173 (dev)
Brain AI Service	Python Flask (PyTorch)	Port 5003
Retinal AI Service	Python Flask (TensorFlow/Keras)	Port 5002
Database	PostgreSQL 14+ via Sequelize v6.37.7	Local PostgreSQL instance
File Storage	Local filesystem	`server/uploads/`
Real-Time	Socket.IO v4.8.1	Attached to Node.js HTTP server
Authentication	JWT via jsonwebtoken v9.0.2	Stateless — no session store needed

Startup Sequence:

1. PostgreSQL must be running — Sequelize connects on server start with `sequelize.sync({ alter: false, force: false })`
2. `cd server && npm run dev` — starts Node.js API server, syncs DB tables
3. `python Brain_Model/app.py` — loads tumor + Alzheimer models (~30 sec startup), serves on port 5003
4. `python Retinal_Model/app.py` — loads EfficientNetB3 model (~30 sec startup), serves on port 5002
5. `cd client && npm run dev` — starts Vite dev server on port 5173

Chapter 5: Testing and Evaluation

Once the system was developed, testing was performed to verify that all implemented modules behave correctly and that the system meets the requirements defined for this implementation. Testing covers unit-level function validation, full feature workflows, business rule enforcement, and cross-component integration.

5.1 Unit Testing

Unit testing verifies individual functions and methods in isolation.

Unit Test 1: validateEmail() — Auth Input Validation

Objective: Verify the email validation function correctly accepts and rejects inputs.

No.	Test Case	Input	Expected Result	Actual Result	Pass/Fail
1	Valid email	`abc@gmail.com`	true	true	Pass
2	Missing @	`abc.gmail.com`	false	false	Pass
3	Empty string	`""`	false	false	Pass
4	Missing domain	`abc@`	false	false	Pass
5	Missing TLD	`abc@domain`	false	false	Pass
6	Subdomain email	`user@mail.example.com`	true	true	Pass
7	Null value	`null`	false	false	Pass

Unit Test 2: validateScan() — File Upload Validation

Objective: Verify scan validation correctly checks file type and size.

No.	Test Case	Input	Expected Result	Actual Result	Pass/Fail
1	Valid JPEG MRI file	`scan.jpg`, 5 MB	Passes validation	[] errors	Pass
2	Invalid file type	`document.txt`	"Invalid file type" error	`["Invalid file type: .txt"]`	Pass
3	Oversized file	`large.jpg`, 60 MB	"Image too large" error	`["Image file too large. Max: 50MB."]`	Pass
4	Valid PNG retinal scan	`retina.png`, 2 MB	Passes validation	[] errors	Pass
5	Null file	`null`	"No file uploaded" error	`["No file uploaded."]`	Pass

Unit Test 3: Password Hashing and Matching (bcrypt)

Objective: Verify password is stored hashed and correctly verified on login.

No.	Test Case	Input	Expected Result	Actual Result	Pass/Fail
1	Create user with password	`"password123"`	Stored as bcrypt hash (not plaintext)	Hash stored in DB	Pass
2	Match correct password	`"password123"` vs stored hash	Returns true	true	Pass
3	Match wrong password	`"wrongpass"` vs stored hash	Returns false	false	Pass
4	Match empty string	`""` vs stored hash	Returns false	false	Pass

---

Unit Test 4: Brain MRI Classification

Objective: Verify the Brain AI Flask service classifies test MRI images correctly.

No.	Test Case	Input	Expected Result	Actual Result	Pass/Fail
1	Glioma MRI image	Glioma brain scan	Returns class "glioma"	glioma	Pass
2	Meningioma MRI	Meningioma scan	Returns class "meningioma"	meningioma	Pass
3	No tumor MRI	Normal brain scan	Returns class "no_tumor"	no_tumor	Pass
4	Pituitary tumor MRI	Pituitary tumor scan	Returns class "pituitary"	pituitary	Pass
5	MildDemented MRI	Alzheimer mild scan	Returns class "MildDemented"	MildDemented	Pass
6	NonDemented MRI	Healthy brain scan	Returns class "NonDemented"	NonDemented	Pass

---

Unit Test 5: GradCAM Heatmap Generation

Objective: Verify GradCAM returns a valid base64 image and region analysis.

No.	Test Case	Input	Expected Result	Actual Result	Pass/Fail
1	Generate heatmap for positive tumor	Glioma MRI	Non-empty base64 string, valid PNG	Valid base64 PNG returned	Pass
2	Region scores present	Any positive MRI	Dict with 5 region names and float scores	5-key dict returned	Pass
3	Top region identified	Glioma MRI	top_region in [frontal lobe, temporal, central, parietal]	Valid region name	Pass
4	Heatmap for no_tumor (negative)	No tumor MRI	Explanation notes no anomaly detected	"No tumor detected" explanation	Pass

5.2 Functional Testing

Functional testing validates that user-facing features work correctly end-to-end.

Functional Test 1: Login with Different Roles

Objective: Verify correct dashboard and navigation bar loads per role.

No.	Test Case	Credentials	Expected	Actual	Pass/Fail
1	Login as Admin	admin@medifusion.com / admin123	Admin dashboard + admin nav	Admin dashboard loaded	Pass
2	Login as Doctor	doctor@test.com / password	Doctor dashboard + doctor nav	Doctor dashboard loaded	Pass
3	Login as Patient	patient@test.com / password	Patient dashboard + patient nav	Patient dashboard loaded	Pass
4	Wrong password	valid email, wrong password	Error toast, stay on login	"Invalid credentials" toast	Pass
5	Inactive account	blocked user credentials	"Account blocked" error	Blocked message returned	Pass

---

Functional Test 2: Patient Books and Pays for Appointment

Objective: Verify full appointment booking with Stripe payment works.

No.	Test Case	Action	Expected	Actual	Pass/Fail
1	Search doctor by specialization	Search "Neurologist"	Matching doctors returned	Doctor list filtered	Pass
2	Select available slot	Click available time	Slot highlights, form populated	Slot selected	Pass
3	Booking with Stripe test card	Card: 4242 4242 4242 4242	Appointment booked, confirmation shown	Appointment created, status = booked	Pass
4	Appointment in My Appointments	Navigate to My Appointments	New appointment visible	Appointment listed	Pass
5	Notification received	After booking	In-app notification created	Bell badge updated	Pass

Functional Test 3: Doctor Runs AI Diagnostics on MRI Scan

Objective: Verify AI analysis pipeline produces correct output.

No.	Test Case	Action	Expected	Actual	Pass/Fail
1	Select uploaded brain scan	Click scan in Diagnostics	Scan preview shown	Preview displayed	Pass
2	Run AI Analysis	Click "Analyze" button	Loading spinner, then results	Results appeared	Pass
3	Tumor prediction shown	Brain MRI with tumor	Class label + confidence %	Correct class + confidence	Pass
4	GradCAM heatmap displayed	After analysis	Heatmap overlaid on image	Heatmap shown	Pass
5	XAI region explanation shown	After analysis	Text explaining which brain region	Region explanation text shown	Pass
6	Alzheimer result shown	Same brain scan	Alzheimer stage + confidence	Alzheimer prediction shown	Pass

Functional Test 4: Admin Manages Users

Objective: Verify admin user management actions work correctly.

No.	Test Case	Action	Expected	Actual	Pass/Fail
1	View all users	Navigate to User Management	Full user list loaded	Users displayed	Pass
2	Search by name	Type in search field	Filtered results shown	List filtered	Pass
3	Filter by role	Select "Doctor" from dropdown	Only doctors shown	Filtered correctly	Pass
4	Block user	Click block icon + confirm	User status changes to blocked	Status → blocked	Pass
5	Unblock user	Click unblock icon + confirm	User status returns to active	Status → active	Pass
6	Delete user	Click delete icon + confirm	User row removed	User deleted from list	Pass

---

Functional Test 5: Virtual Consultation with AI Notes

Objective: Verify Jitsi video integration and AI note generation work together.

No.	Test Case	Action	Expected	Actual	Pass/Fail
1	Join consultation room	Navigate to ConsultationRoom	Jitsi video iframe loads	Video call interface shown	Pass
2	Send chat message	Type and send message	Message appears in chat log	Message visible to both parties	Pass
3	Record audio	Click "Start Recording"	Recording indicator shown	Recording started	Pass
4	Generate notes	Click "Stop & Generate"	AI processes audio, structured notes shown	Notes generated with all fields	Pass
5	Edit generated notes	Modify fields in note modal	Changes saved	Updated content retained	Pass
6	Finalize notes	Click "Finalize"	Notes locked, patient summary unlocked	Note finalized	Pass

5.3 Business Rules Testing

Business rules testing verifies system constraints and domain-specific rules.

No.	Business Rule	Test	Expected	Actual	Pass/Fail
1	Doctor cannot book own appointment	Doctor tries to book with themselves	Rejected by backend	Booking prevented	Pass
2	Only admin can verify doctors	Patient/doctor calls admin endpoint	403 Forbidden	403 returned	Pass
3	Patient can only view own scans	Patient requests another patient's scan ID	403 Forbidden	Access denied	Pass
4	Appointment must be paid before it is booked	Create appointment without payment	Status stays pending_payment	Not promoted to booked	Pass
5	Doctor cannot see admin dashboard	Doctor navigates to /admin	Redirected to doctor dashboard	Redirect applied	Pass
6	Finalized consultation notes are read-only	Edit finalized note	Edit blocked	Cannot modify finalized note	Pass
7	Scan file must be JPEG or PNG	Upload .pdf scan	"Invalid file type" error	Rejected by Multer	Pass
8	Doctor approval required before login	Pending doctor tries to log in	"Account pending verification"	Login blocked	Pass

5.4 Integration Testing

Integration testing verifies that cross-component communication works correctly.

Integration Test 1: Backend ↔ Brain AI Flask Service

Objective: Verify Node.js backend correctly communicates with Brain Model on port 5003.

No.	Test	Expected	Actual	Pass/Fail
1	POST `/api/scans/analyze-brain` with valid MRI	Brain Flask returns tumor + Alzheimer JSON	Full AI result JSON received	Pass
2	Flask service down	Backend returns 503 "AI service unavailable"	Graceful error response	Pass
3	GradCAM heatmap in response	Scan analysis response contains base64 field	base64 string present in `gradcam_heatmap`	Pass
4	AI result stored in Scan record	After analysis, fetch scan by ID	`ai_result` and `gradcam_heatmap` fields populated	Pass

---

Integration Test 2: Backend ↔ Retinal AI Flask Service

No.	Test	Expected	Actual	Pass/Fail
1	POST `/api/scans/analyze` with retinal image	Retinal Flask returns disease class + heatmap	Correct class + base64 heatmap returned	Pass
2	Zone analysis included	Response contains `zone_scores` and `is_expected`	Region analysis fields present	Pass
3	All 4 class probabilities returned	Response contains `all_probs` dict	4-key probability dict present	Pass

---

Integration Test 3: Backend ↔ Stripe ↔ Database

No.	Test	Expected	Actual	Pass/Fail
1	PaymentIntent created in Stripe	`client_secret` returned to frontend	Valid Stripe client secret returned	Pass
2	Webhook fires on test payment success	`charge.succeeded` event received	Payment record updated to succeeded	Pass
3	Appointment status updated on payment	After webhook	Appointment.status = 'booked'	Pass
4	Payment record created in DB	After webhook	Payment + PaymentTransaction rows created	Pass

---

Integration Test 4: Backend ↔ Groq AI (AI Note-Taker)

No.	Test	Expected	Actual	Pass/Fail
1	Audio file transcribed via Whisper	Transcript text returned	Text returned from Groq API	Pass
2	Urdu words translated in transcript	Mixed Urdu/English audio	All text in English in structured notes	Pass
3	Structured notes contain all fields	LLaMA response parsed	`chiefComplaint`, `HPI`, `assessment`, `plan`, `patientSummary` all present	Pass
4	Notes saved to ConsultationNote model	After generation	ConsultationNote row created with transcript and structured_notes JSON	Pass

---

Integration Test 5: Frontend ↔ Backend (Socket.IO Chat)

No.	Test	Expected	Actual	Pass/Fail
1	Patient and doctor join same room	Both connected to `appointment-{id}` socket room	Both receive messages	Pass
2	Message sent by doctor	Patient receives message in real-time	Message appears without page refresh	Pass
3	Chat history persisted	Reload consultation page	Previous messages loaded from ChatLog table	Pass